

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

старший преподаватель
должность, уч. степень, звание

подпись, дата

Д. О. Шевяков
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №5

Создание и настройка системы
управления оборудованием на базе
платформы интернета вещей

по курсу: Интернет вещей

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

Г.В. Буренков
инициалы, фамилия

Санкт-Петербург 2026

СОДЕРЖАНИЕ

1 Цель работы	2
2 Задание	3
3 Ход выполнения задания.....	4
4 Вывод.....	7
ПРИЛОЖЕНИЕ А	8

1 Цель работы

Целью лабораторной работы является получение практических навыков валидации входных параметров на стороне сервера, приведения данных к корректному формату и отклонения некорректных управляющих команд без изменения состояния устройств.

2 Задание

В соответствии с методическими указаниями для лабораторной работы №5 требуется модифицировать функции управления вещами так, чтобы все входящие значения проверялись перед применением. Необходимо использовать как минимум два формата входных данных, включая строковый формат, и применить хотя бы одно регулярное выражение.

3 Ход выполнения задания

В ходе выполнения лабораторной работы сохранена архитектура лабораторной работы №4: мониторинг выполняется через маршруты `GET /connect/temperature`, `GET /connect/light`, `GET /connect/lock`, а управление — через `GET /command/temperature`, `GET /command/light`, `GET /command/lock`. Отличие ЛР5 заключается в том, что перед вызовом `applyCommand` для каждой вещи добавлена серверная проверка входных значений. На рисунке 1 изображена страница приложения с результатами успешных и отклонённых команд.

Мониторинг

Температура: —

Свет: —, яркость —%

Замок: —

Управление (с валидацией на сервере)

Температура

Примеры: 23.5 или 23,5

{"ok":false,"error":"Строка не соответствует формату числа (допустимы цифры, опционально «.» или «,» как разделитель дробной части)"}

Свет

— слово: on, off, true, false, да, нет ... Яркость — только цифры 0–100.

Замок

Строка из словаря: , , , , ...

Рисунок 1 - Скриншот браузера: видны блоки управления и сообщения об ошибках валидации.

Для температуры реализована проверка строкового значения регулярным выражением (допускаются форматы с точкой и запятой), затем выполняется преобразование к числу с контролем `NaN`. Для света проверяются два параметра: логическое состояние (`enabled`) по словарю допустимых строк и яркость (`brightness`) как целое число в диапазоне 0-100. Для замка параметр `locked` проверяется по набору допустимых строковых представлений состояния. При некорректных данных сервер возвращает код 400 и JSON с полем `error`, а состояние устройств не изменяется. На рисунке 2 изображена вкладка «Сеть» с примерами отклонённых запросов.

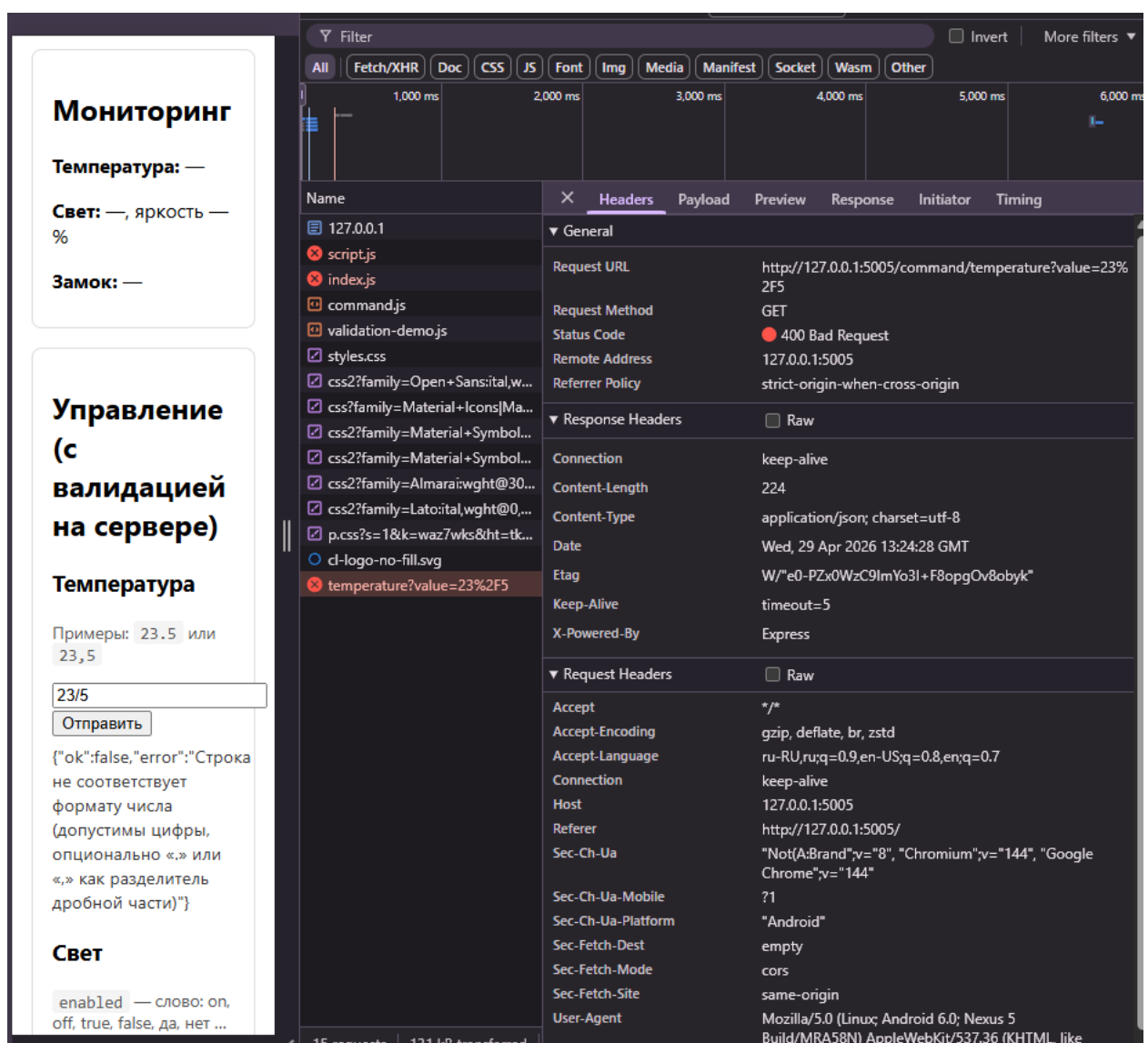


Рисунок 2 - Скриншот Network: видны запросы `/command/...` с кодом 400 и текстом ошибки.

В консоли сервера логируются обращения к маршрутам и причины отклонения неверных значений, что позволяет наглядно подтвердить работу механизма валидации. На рисунке 3 изображён фрагмент журнала терминала при отправке ошибочных команд.

```
> lab5@1.0.0 start
> tsx src/app.ts

[LightController.setLight] light-1
[SmartLock.setLocked] lock-1
http://127.0.0.1:5005/ - /IP5 (валидация входящих данных)
[2026-04-29T13:23:27.465Z] GET /command.js
[2026-04-29T13:23:35.370Z] GET /command/temperature
[TemperatureSensor.applyCommand] отклонено: Строка не соответствует формату числа (допустимы цифры, опционально «.» или «,» как разделитель дробной части)
```

Рисунок 3 - Скриншот терминала с логами валидации.

4 Вывод

В результате выполнения лабораторной работы реализована серверная валидация входных данных для всех устройств варианта «Умный дом» с использованием регулярных выражений, словарей допустимых значений и проверки диапазонов. Полученная реализация исключает применение некорректных команд и подготавливает систему к дальнейшему развитию в следующих лабораторных работах.

ПРИЛОЖЕНИЕ А

Листинг с кодом программы

В данном разделе представлен исходный код моей части программы:

```
// валидация validation.ts
function showBad(url) {
  const out = document.getElementById("badDemoOut");
  fetch(url)
    .then(async (r) => {
      const text = await r.text();
      try {
        const j = JSON.parse(text);
        out.textContent = `HTTP ${r.status}\n${JSON.stringify(j, null, 2)}`;
      } catch {
        out.textContent = `HTTP ${r.status}\n${text}`;
      }
    })
    .catch((e) => {
      out.textContent = String(e);
    });
}

document.getElementById("badTemp").addEventListener("click", () => {
  showBad("/command/temperature?value=abc");
});

document.getElementById("badLight").addEventListener("click", () => {
  showBad("/command/light?enabled=maybe&brightness=50");
});
```

```
document.getElementById("badLock").addEventListener("click", () => {  
    showBad("/command/lock?locked=скоpee_нет");  
});
```

```
// app.ts
```

```
import express from "express";
```

```
import path from "node:path";
```

```
import { TemperatureSensor, LightController, SmartLock } from "./things";
```

```
const app = express();
```

```
const PORT = 5000;
```

```
const temp = new TemperatureSensor("sensor-1", "Температурный датчик", 22.5);
```

```
const light = new LightController("light-1", "Свет в гостиной");
```

```
const lock = new SmartLock("lock-1", "Замок входной двери");
```

```
light.setLight(true, 65);
```

```
lock.setLocked(true);
```

```
app.use((req, _res, next) => {
```

```
    if (req.path.startsWith("/connect") || req.path.startsWith("/command")) {
```

```
        console.log(`[${new Date().toISOString()}] ${req.method} ${req.path}`);
```

```
    }
```

```
    next();
```

```
});
```

```
app.use(express.static(path.join(process.cwd(), "public")));
```

```
app.get("/connect/temperature", (_req, res) => res.json(temp.connect()));
```

```
app.get("/connect/light", (_req, res) => res.json(light.connect()));
app.get("/connect/lock", (_req, res) => res.json(lock.connect()));
```

```
app.get("/command/temperature", (req, res) => {
  const q = req.query as Record<string, string | undefined>;
  const result = temp.applyCommand(q);
  if (!result.ok) {
    return res.status(400).json(result);
  }
  return res.json({ ok: true, state: temp.snapshot() });
});
```

```
app.get("/command/light", (req, res) => {
  const q = req.query as Record<string, string | undefined>;
  const result = light.applyCommand(q);
  if (!result.ok) {
    return res.status(400).json(result);
  }
  return res.json({ ok: true, state: light.snapshot() });
});
```

```
app.get("/command/lock", (req, res) => {
  const q = req.query as Record<string, string | undefined>;
  const result = lock.applyCommand(q);
  if (!result.ok) {
    return res.status(400).json(result);
  }
  return res.json({ ok: true, state: lock.snapshot() });
});
```

```
app.listen(PORT, () => {  
  console.log(`http://127.0.0.1:${PORT}/`);  
});
```

```
//things.ts
```

```
export type CommandResult = { ok: true } | { ok: false; error: string };
```

```
export abstract class Thing {
```

```
  constructor(  
    public id: string,  
    public name: string,  
    public isOnline: boolean = true  
  ) {}
```

```
  protected abstract emulate(): void
```

```
  protected abstract buildPayload(): Record<string, unknown>;
```

```
  connect(): Record<string, unknown> {
```

```
    console.log(`[Thing.connect] ${this.constructor.name} (${this.id})`);  
    this.emulate();  
    return this.buildPayload();  
  }
```

```
  /** Снимок состояния без эмуляции (после команды). */
```

```
  snapshot(): Record<string, unknown> {  
    return this.buildPayload();  
  }
```

```

    abstract getStatus(): string;
}

export interface IParent {
    render(statuses: string[]): string;
}

export interface IChild {
    render(statuses: string[]): string;
}

export class MainControlUnit {
    constructor(
        public readonly things: Thing[],
        public readonly datastore: DeviceDataStore
    ) {}

    registerThing(thing: Thing): void {
        console.log(`[MainControlUnit.registerThing] ${thing.id}`);
        this.things.push(thing);
    }

    collectMonitoringData(): void {
        console.log(`[MainControlUnit.collectMonitoringData]`);
        for (const thing of this.things) {
            const line = `[${new Date().toISOString()}] ${thing.getStatus()}`
            this.dataStore.saveRecord(thing, line)
        }
    }

    getAllStatuses(): string[] {
        console.log(`[MainControlUnit.getAllStatuses]`);

```

```

        return this.things.map((t) => t.getStatus());
    }
}

export class DeviceDataStore {
    private readonly dataByThingId = new Map<string, string[]>();

    saveRecord(thing: Thing, payload: string): void {
        console.log(`[DeviceDataStore.saveRecord] ${thing.id}`);
        const list = this.dataByThingId.get(thing.id) ?? [];
        list.push(payload);
        this.dataByThingId.set(thing.id, list);
    }

    getRecords(thingId: string): string[] {
        console.log(`[DeviceDataStore.getRecords] ${thingId}`);
        return this.dataByThingId.get(thingId) ?? [];
    }
}

export class TemperatureSensor extends Thing {
    private currentTemperatureC: number;

    constructor(id: string, name: string, initialTemperatureC: number) {
        super(id, name, true);
        this.currentTemperatureC = initialTemperatureC;
    }

    setTemperature(valueC: number): void {

```

```

    console.log(`[TemperatureSensor.setTemperature] ${this.id}`);
    this.currentTemperatureC = valueC;
}

```

```

getTemperature(): number {
    return this.currentTemperatureC;
}

```

*/** IP4: параметры из query (аналог request.args). */*

```

applyCommand(query: Record<string, string | undefined>): CommandResult {
    const raw = query.value ?? query.temperature;
    if (raw === undefined || raw === "") {
        return { ok: false, error: "Ожидается параметр value (или temperature)" };
    }
    const n = Number(raw);
    if (Number.isNaN(n)) {
        return { ok: false, error: "value должно быть числом" };
    }
    this.setTemperature(n);
    console.log(`[TemperatureSensor.applyCommand] ${this.id} -> ${n}`);
    return { ok: true };
}

```

```

protected emulate(): void {
    const delta = (Math.random() - 0.5) * 1.2;
    this.currentTemperatureC = Math.round((this.currentTemperatureC + delta) *
10) / 10;
}

```

```

protected buildPayload(): Record<string, unknown> {

```

```

    return {
      id: this.id,
      name: this.name,
      value: this.currentTemperatureC,
      unit: "°C"
    };
  }

  getStatus(): string {
    console.log(`[TemperatureSensor.getStatus] ${this.id}`);
    return `${this.name}: ${this.currentTemperatureC.toFixed(1)} °C`
  }
}

export class LightController extends Thing {
  private brightnessPercent: number = 0;
  private isEnabled: boolean = false;

  setLight(isEnabled: boolean, brightnessPercent: number): void {
    console.log(`[LightController.setLight] ${this.id}`);
    this.isEnabled = isEnabled;
    this.brightnessPercent = Math.max(0, Math.min(100, brightnessPercent))
  }

  applyCommand(query: Record<string, string | undefined>): CommandResult {
    const enabledRaw = query.enabled ?? query.on;
    const brightRaw = query.brightness ?? query.brightnessPercent;
    if (enabledRaw === undefined || brightRaw === undefined || brightRaw ===
    "" ) {

```



```

        return { ok: false, error: "Ожидаются enabled (true/false) и brightness (0–
100)" };
    }
    const enabled = enabledRaw === "true" || enabledRaw === "1";
    const brightness = Number(brightRaw);
    if (Number.isNaN(brightness)) {
        return { ok: false, error: "brightness должно быть числом" };
    }
    this.setLight(enabled, brightness);
    console.log(`[LightController.applyCommand] ${this.id} enabled=${enabled}
brightness=${brightness}`);
    return { ok: true };
}

```

```

protected emulate(): void {
    const delta = Math.floor((Math.random() - 0.5) * 12);
    this.brightnessPercent = Math.max(0, Math.min(100, this.brightnessPercent +
delta));
}

```

```

protected buildPayload(): Record<string, unknown> {
    return {
        id: this.id,
        name: this.name,
        enabled: this.isEnabled,
        brightnessPercent: this.brightnessPercent
    };
}

```

```

getStatus(): string {

```

```

        console.log(`[LightController.getStatus] ${this.id}`);
        return `${this.name}: ${this.isEnabled ? "ВКЛ" : "ВЫКЛ"},
яркость=${this.brightnessPercent}%`;
    }
}

```

```

export class SmartLock extends Thing{
    private isLocked: boolean = true;

```

```

    setLocked(value: boolean): void {
        console.log(`[SmartLock.setLocked] ${this.id}`);
        this.isLocked = value;
    }

```

```

    applyCommand(query: Record<string, string | undefined>): CommandResult {
        const raw = query.locked ?? query.lock;
        if (raw === undefined || raw === "") {
            return { ok: false, error: "Ожидается locked (true/false)" };
        }
        if (raw !== "true" && raw !== "false" && raw !== "1" && raw !== "0") {
            return { ok: false, error: "locked должно быть true или false" };
        }
        const locked = raw === "true" || raw === "1";
        this.setLocked(locked);
        console.log(`[SmartLock.applyCommand] ${this.id} locked=${locked}`);
        return { ok: true };
    }

```

```

    protected emulate(): void {
        if (Math.random() < 0.08) this.isLocked = !this.isLocked;

```

```
}
```

```
protected buildPayload(): Record<string, unknown> {
```

```
  return {
```

```
    id: this.id,
```

```
    name: this.name,
```

```
    locked: this.isLocked
```

```
  };
```

```
}
```

```
getStatus(): string {
```

```
  console.log(`[SmartLock.getStatus] ${this.id}`);
```

```
  return `${this.name}: ${this.isLocked ? "ЗАКРЫТ" : "ОТКРЫТ"}`;
```

```
}
```

```
}
```

```
export class ParentInterface implements IParent {
```

```
  render(statuses: string[]): string {
```

```
    console.log("[ParentInterface.render]");
```

```
    return ["Интерфейс родителя (полный доступ)", ...statuses].join("\n");
```

```
  }
```

```
}
```

```
export class ChildInterface implements IChild {
```

```
  render(statuses: string[]): string {
```

```
    console.log("[ChildInterface.render]");
```

```
    return ["Интерфейс ребенка (ограничен)", ...statuses].join("\n");
```

```
  }
```

```
}
```

```

export function runDemo(): {
  parentText: string;
  childText: string;
  sensorRecords: string[];
} {
  const store = new DeviceDataStore();
  const things: Thing[] = [];
  const mcu = new MainControlUnit(things, store);
  const temp = new TemperatureSensor("sensor-1", "Температурный датчик",
22.5);
  const light = new LightController("light-1", "Свет в гостиной");
  const lock = new SmartLock("lock-1", "Замок входной двери");
  light.setLight(true, 65);
  lock.setLocked(true);
  mcu.registerThing(temp);
  mcu.registerThing(light);
  mcu.registerThing(lock);
  mcu.collectMonitoringData();
  const statuses = mcu.getAllStatuses();
  const parentText = new ParentInterface().render(statuses);
  const childText = new ChildInterface().render(statuses);
  return {
    parentText,
    childText,
    sensorRecords: store.getRecords("sensor-1")
  };
}

```

```
//script.js
```

```
const CONNECT = {
```

```

    temperature: "/connect/temperature",
    light: "/connect/light",
    lock: "/connect/lock",
};

```

```

async function fetchJson(url) {
    const response = await fetch(url);
    if (!response.ok) {
        throw new Error(`HTTP ${response.status} для ${url}`);
    }
    return response.json();
}

```

```

//index.js
function $(id) {
    return document.getElementById(id);
}

```

```

async function updateTemperature() {
    const data = await fetchJson(CONNECT.temperature)
    document.getElementById("tempValue").textContent = `${data.value}
    ${data.unit}`
}

```

```

async function updateLight() {
    const data = await fetchJson(CONNECT.light);
    $("lightEnabled").textContent = data.enabled ? "ВКЛ" : "ВЫКЛ";
    $("lightBrightness").textContent = String(data.brightnessPercent);
}

```

```

async function updateLock() {
  const data = await fetchJson(CONNECT.lock);
  $("lockState").textContent = data.locked ? "ЗАКРЫТ" : "ОТКРЫТ";
}

async function tick() {
  await Promise.all([updateTemperature(), updateLight(), updateLock()]);
}

```

```

tick();
setInterval(tick, 1000);

```

```

//index.html
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>IoT Дом — ЛР4</title>
  <style>
    body { font-family: system-ui, sans-serif; max-width: 42rem; margin: 1.5rem
auto; padding: 0 1rem; }
    section { border: 1px solid #ddd; border-radius: 8px; padding: 0.75rem 1rem;
margin: 1rem 0; }
    h1 { font-size: 1.35rem; }
    code { background: #f4f4f4; padding: 0.1rem 0.35rem; border-radius: 4px; }
    .result { font-size: 0.9rem; color: #333; margin-top: 0.35rem; }
  </style>
</head>
<body>
  <h1>ЛР4 — мониторинг (ЛР3) + команды управления</h1>

```

<section>

<h2>Мониторинг</h2>

<p>Обновление раз в секунду через <code>GET /connect/...</code></p>

<p>Температура: —</p>

<p>Свет: —, яркость
—%</p>

<p>Замок: —</p>

</section>

<section>

<h2>Управление</h2>

<p>Отправка на <code>GET /command/...</code> с параметрами в строке
запроса.</p>

<div>

<h3>Температура</h3>

<input id="cmdTemp" type="text" placeholder="например 23.5" />

<button id="btnTemp" type="button">Отправить</button>

<div class="result" id="cmdTempResult"></div>

</div>

<div>

<h3>Свет</h3>

<label><input id="cmdLightOn" type="checkbox" checked />
Включено</label>

<input id="cmdLightBright" type="number" min="0" max="100" value="65"
/>

<button id="btnLight" type="button">Отправить</button>

<div class="result" id="cmdLightResult"></div>

```

</div>

<div>
  <h3>Замок</h3>
  <label><input id="cmdLockLocked" type="checkbox" checked />
  Закрыт</label>
  <button id="btnLock" type="button">Отправить</button>
  <div class="result" id="cmdLockResult"></div>
</div>
</section>

<script src="/script.js"></script>
<script src="/index.js"></script>
<script src="/command.js"></script>
</body>
</html>

```

```

//command.js
async function sendCommand(url) {
  const response = await fetch(url);
  const data = await response.json();
  if (!response.ok) {
    throw new Error(JSON.stringify(data));
  }
  return data;
}

```

```

function setResult(id, text) {
  const el = document.getElementById(id);
  if (el) el.textContent = text;
}

```



```
}
```

```
document.getElementById("btnTemp").addEventListener("click", async () => {  
  const value = document.getElementById("cmdTemp").value.trim();  
  try {  
    const data = await sendCommand(  
      `/command/temperature?value=${encodeURIComponent(value)}`  
    );  
    setResult("cmdTempResult", JSON.stringify(data));  
  } catch (e) {  
    setResult("cmdTempResult", String(e.message));  
  }  
});
```

```
document.getElementById("btnLight").addEventListener("click", async () => {  
  const enabled = document.getElementById("cmdLightOn").checked;  
  const brightness = document.getElementById("cmdLightBright").value;  
  const qs = new URLSearchParams({  
    enabled: String(enabled),  
    brightness: String(brightness)  
  });  
  try {  
    const data = await sendCommand(`/command/light?${qs.toString()}`);  
    setResult("cmdLightResult", JSON.stringify(data));  
  } catch (e) {  
    setResult("cmdLightResult", String(e.message));  
  }  
});
```

```
document.getElementById("btnLock").addEventListener("click", async () => {
```

```
const locked = document.getElementById("cmdLockLocked").checked;
const qs = new URLSearchParams({ locked: String(locked) });
try {
    const data = await sendCommand(`/command/lock?${qs.toString()}`);
    setResult("cmdLockResult", JSON.stringify(data));
} catch (e) {
    setResult("cmdLockResult", String(e.message));
}
});
```